people.c — Betegnyilvántartó Program

Biztonsági Sérülékenységek Elemzése

SZTE Információbiztonság kurzus — C - B esszé

Felhasznált AI prompt dokumentáció

A feladat elemzéséhez és az esszé vázlatának elkészítéséhez ezeket a Claude Sonnet 4.6-os promptokat használtam. Az AI által generált elemzések, megállapítások ellenőrzése és végleges jóváhagyása a saját munkám. Általánosságban elmondhatom, hogy az AI segítette a sérülékenységek pontos helyének feltárását, működésük megértését és a PoC-k elkészítését.

A javításokkal kiegészített fork itt érhető el: https://github.com/CaTwoPlus/people

---

1. Fordítói figyelmeztetések

A program `gcc people.c -o p -Wall` paranccsal fordítva az alábbi 7 figyelmeztetést adja (a fordítóban a linker-szintű gets-figyelmeztetéssel együtt). A táblázat minden egyes figyelmeztetés okát és javasolt javítását mutatja be. Megjegyzés: az árnyékolási problémák (változónév-shadowing) a `-Wall`-ban nem jelennek meg — azokat a `-Wshadow` kapcsoló hozná elő, és a 2. fejezetben tárgyaljuk.

Figyelmeztetés	Ok	Javítás
`implicit declaration of 'gets'`	A gets() az ISO C11 szabványból eltávolításra került. A fordító implicit deklarációként kezeli.	Cseréljük fgets(buf, sizeof(buf), stdin) hívásra.
`unused variable 'm', 'n' (main)`	A main()-ben deklarált int m,n; lokális változók soha nem kerülnek felhasználásra — felesleges deklarációk.	Töröljük mindkét változót; a globális n-t az add() már közvetlenül használja.
`unused variable 'i', 'q' (main)`	A main()-ben deklarált int c,i,q; közül i és q sohasem kerülnek felhasználásra.	Töröljük a felesleges deklarációkat.
`unused variable 'f' (search)`	A search() külső blokkjában deklarált int s,h,f; közül f soha nem kerül értékadásra ott, ahol deklarálva van.	Töröljük az outer-scope f-et; a belső blokkok saját f-et deklarálnak.
`'return' with value in void function (read)`	A void read() végén return 0; áll, ami értéket ad vissza void függvényből.	Cseréljük return; (érték nélkül) hívásra.

Miért fontosak ezek a figyelmeztetések?

A fordítói figyelmeztetések közvetlen utalást adnak a kód potenciálisan veszélyes részeire. A `gets()` implicit deklarációja például nem csupán stílusbeli probléma: jelzi, hogy a függvény teljesen el lett távolítva a C11 standard könyvtárából. A linker csak azért fogadja el, mert a glibc visszafelé kompatibilis — de a viselkedése továbbra is kritikus biztonsági kockázatot jelent (ld. 3a. fejezet). A void függvénybeli return 0 és az inicializálatlan változókat a fejlettebb eszközök (Valgrind) is jelzik, ahogy azt az 5. fejezetben látjuk majd.

Konkrét javítási javaslat

read() void függvény visszatérési értéke:

```
401            return 0;
```

```
407        return;
```

Felesleges változók törlése:

```
19  int main()
20  {
21      read();
22      int c,i,q;
23      printf("========== Patient Recording System ===========\n");
24      int m,n;
```

```
19  int main()
20  {
21      read();
22      int c = 0;
23      printf("========== Patient Recording System ===========\n");
```

```
180  void search()
181  {
182      int s,h,f;
```

```
180  void search()
181  {
182      int s,h;
```

A gets() fgets()-re való cseréje:

```
83      gets(x[i].name);
```

```
83      fgets(patients[i].name, sizeof(patients[i].name), stdin);
```

2. Kód formázás és olvashatóság

Behúzások, sortörések, kommentek

A sorok behúzása nagyrészt rendben van. Helyenként fölösleges sortörések vannak, míg máshol hiányzik sortörés. Kommenteket nem előzi meg whitespace, vagy nem az utasítás fölött vannak, így egybefolynak az utasításokkal. A kód ilyen szempontból is javítva lett.

Változónevek értelmezhetősége

A kód tele van egybetűs és rövidített globális változónevekkel, amelyek az olvasónak semmilyen információt nem adnak a változó szerepéről. Az alábbi táblázat a legproblematikusabb eseteket mutatja be:

Változó	Javasolt név	Megjegyzés
`a`	`(törölhető)`	Hozzáadott rekordok száma a jelenlegi session-ben — teljesen felesleges, num tölti be ezt a szerepet.
`num`	`recordCount`	-
`g`	`searchIndex`	Csak a search() ciklus-indexeként szerepel — a globális i mintájára, de ott sem indokolt a globalitás.
`flag`	`(törölhető)`	Deklarálva van, de soha nem kerül felhasználásra a programban.
`j`	`(törölhető)`	Növelése párhuzamosan fut i-vel és a-val az add()-ban, de az értékét sehol nem olvassák.
`sum`	`endIndex`	Az add() for-ciklusának felső határát jelöli, de a neve nem utal erre.
`x[100]`	`patients[100]`	A globális struct tömb neve egy egybetűs x — a patients[] egyértelműbb lenne.
`struct ad`	`struct Patient`	Az adatszerkezet neve semmilyen információt nem ad a tartalomról.

Egyéb karbantarthatósági problémák

- **show() definiálatlan:** A függvény deklarálva van (void show();), de definiálva és meghívva sehol sem — a fordítónak ez nem okoz hibát, de félrevezeti az olvasót.
- **Globális változók mindenhol:** A n, i, j, a, sum, g, flag, num változók mind globálisak, holott legtöbbjük csak egy-egy függvényen belül szükséges. Ez nehezíti a megértést és elrejti a logikai hibákat.
- **Mágikus számok:** A 100-as kapacitáskorlátot és a structban lévő mezők hosszát (30) jobb lenne előfeldolgozó makrókkal definiálni (pl. #define MAX_PATIENTS 100), így változás esetén könnyen módosíthatóak az értékek, illetve egyértelmű hogy az értékek mit reprezentálnak.
- **Üres default ágak:** Az edit() és del() függvények nem kezelik a negatív serial szám esetét (p < 0 ill. f < 0), csak az if(p<num) feltétel alapján dönt. Javítása:

```
136        if(p >= 0 && p < num)
```

```
347        if(f >= 0 && f < num)
```

3. Sérülékeny kódrészletek azonosítása

3a. Buffer Overflow — gets() korlátlan bemenet

A legsúlyosabb sérülékenység a `gets()` függvény használata. A struct mezők `name[30]` és `disease[30]` mindössze 30 karaktert tudnak tárolni, de a `gets()` addig olvas, amíg sortörést vagy EOF-ot nem kap — határellenőrzés nélkül.

```
// add() függvényből — sérülékeny sor:
// name mérete: 30 byte, de bemenet lehet akármennyi
83            gets(x[i].name);
// A struct ad memóriaelrendezése:
// [ name[30] | disease[30] | cabin(4) | phone(4) | age(4) ] = 72 byte/rekord
// x[100] → összesen 7200 byte statikusan a BSS szegmensben
```

Ha a felhasználó 30 karakternél hosszabb nevet ad meg, az adat *túlcsordul* a `disease`, `cabin`, `phone`, `age` mezőkre, és potenciálisan a szomszédos x[i+1] struktúra elejére is. Mivel az x tömb globális (BSS szegmens), a stack ugyan nem kerül közvetlenül veszélybe, de a többi páciens adatai felülíródnak, az adatintegritás sérül. Modósításnál

Proof of concept bemenet:

```
# 50 'A' karaktert ír a name mezőbe (limit: 30):
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
# Eredmény: x[i].disease tartalma felülíródik 'AAAAAAAAAAA...'-val
```

```
Serial Number=0
Name = teszt
Disease = teszt
Cabin no = 0
Phone number = 00
Age=0


Serial Number=1
Name = teszt
Disease = teszt
Cabin no = 0
Phone number = 00
Age=0
```

```
Serial Number=0
Name = AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Disease = AAAAAAAAAAAAAAAAAAA
Cabin no = 0
Phone number = 00
Age=0


Serial Number=1
Name = teszt
Disease = teszt
Cabin no = 0
Phone number = 00
Age=0
```

Javítás — fgets() korlátozott olvasás:

```
// Biztonságos:
85        fgets(patients[i].name, sizeof(patients[i].name), stdin);
// Az fgets() legfeljebb sizeof-1 karaktert olvas, majd \0-t ír;
// a sortörést is megőrzi a bufferben — azt kézzel el kell távolítani:
86        patients[i].name[strcspn(patients[i].name, "\n")] = '\0';
```

```
Serial Number=0
Name = teszt
Disease = teszt
Cabin no = 0
Phone number = 00
Age=0


Serial Number=1
Name = elek
Disease = elek
Cabin no = 0
Phone number = 00
Age=0
```

```
What do you want to edit ?
1.Name
2.Disease
3.Age
4.Cabin
5.Phone no.
Option=1
Enter the serial no of that patient (0 - 1): 0
Enter the new name=AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

```
Serial Number=0
Name = AAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Disease = teszt
Cabin no = 0
Phone number = 00
Age=0


Serial Number=1
Name = elek
Disease = elek
Cabin no = 0
Phone number = 00
Age=0
```

3b. Inicializálatlan változó — while(c != 6) feltétel

A main() elején deklarált `int c` változó nem kap kezdőértéket, mégis azonnal feltételként szerepel a while ciklusban. Ez **meghatározatlan viselkedés (undefined behavior)** a C szabvány szerint.

```
19  int main()
20  {
21      read();
22      int c,i,q;
```

```
26      while(c!=6)
```

A Valgrind memóriaelemző erre explicit hibát jelez:

```
razor@ReBorn:/mnt/e/source/repos/SZTE/Informacio_bizt/people$ valgrind --track-origins=yes ./o
==760== Memcheck, a memory error detector
==760== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==760== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==760== Command: ./o
==760==
========== Patient Recording System ===========
==760== Conditional jump or move depends on uninitialised value(s)
==760==    at 0x1094A9: main (in /mnt/e/source/repos/SZTE/Informacio_bizt/people/o)
==760==  Uninitialised value was created by a stack allocation
==760==    at 0x109329: main (in /mnt/e/source/repos/SZTE/Informacio_bizt/people/o)
==760==

**Enter your choice**

1. Add Patient
2. View Patient Record
3. Search Patient
4. Update Patient
5. Delete Patient
6. Exit

Option=
```

A while belépési feltétele az első `scanf` hívás előtt kerül kiértékelésre. Ha `c` véletlenszerűen `6`-ot tartalmaz a stack-en, a program azonnal kilép anélkül, hogy a menüt megjelenítette volna. Fordítva: ha a fordító valamilyen értékkel feltölti a stacket, a program végtelen ciklust futhat az első scanf előtt.

Javítás:

```
// Biztonságos:
```

```
22  int main()
23  {
24      read();
25      int c = 0;
26  >   do {···
67      } while(c!=6);
68  }
```

```
// A do-while garantálja, hogy a menü legalább egyszer megjelenik,
// és c első értéke scanner-ből jön, nem a stack-ről.
```

3c. Tömbhatáron túli írás — del() utolsó rekord törlése

A `del()` függvény az adott sorszámú rekordot úgy törli, hogy az összes mögötte lévő elemet egy pozícióval előre másolja. A while ciklus feltétele `f < num`, de az utolsó lépésben — amikor `f == num-1` — a kód `x[f+1]` azaz `x[num]` tartalmát olvassa. Ez egy 100 elemű tömbön **x[100] hozzáférés**, ami a C standard szerint meghatározatlan viselkedés és memóriakorrupciót okozhat.

```c
void del()
{
    int f,h;
    printf("Enter the serial number of the patient that you want to delete=");
    scanf("%d",&f);
    if(f<num)
    {
        printf("What do you want ?\n");
        printf("1.Remove the whole record\n2.Remove Name\n3.Remove Disease\n4.Remove age\n5.Remove Cabin\n6.Remove phone number\nOption = ");
        scanf("%d",&h);
        if(h==1)
        {
            while(f<num) // utolsó körben f == num-1
            {
                strcpy(x[f].name,x[f+1].name); // x[num] → OOB!
                strcpy(x[f].disease,x[f+1].disease);
                x[f].age=x[f+1].age;
                x[f].cabin=x[f+1].cabin;
                x[f].phone=x[f+1].phone;
                f++;
            }
            num--;
        }
```

Javítás — a ciklus feltételét num-1-re kell módosítani:

```c
            while(f < num - 1) // az utolsó valid index: num-2 → f+1 = num-1
            {
                strcpy(patients[f].name,patients[f+1].name);
                strcpy(patients[f].disease,patients[f+1].disease);
                patients[f].age=patients[f+1].age;
                patients[f].cabin=patients[f+1].cabin;
                patients[f].phone=patients[f+1].phone;
                f++;
            }
            // Az utolsó elemet nullázzuk ki (opcionális, de jó gyakorlat)
            memset(&patients[num-1], 0, sizeof(struct Patient));
            num--;
        }
```

3d. Tömbkapacitás túllépése — korlátlan páciensfelvétel

Az `x[100]` globális tömb legfeljebb 100 pácienst tárolhat. Az `add()` függvény azonban semmilyen ellenőrzést nem végez arra, hogy `sum = n + num` ne haladja meg a 100-at. Ha például `num == 95` és a felhasználó 10 új rekordot kér, a program az `x[95]..x[104]` indexekre ír, ami **tömbhatáron túli memóriaírás**

```c
    sum=n+num;  // nincs ellenőrzés, hogy sum <= 100!

    for(i=num,j=0; i<sum; i++)
    {
        printf("\n");
        printf("Enter patient's Name = ");
        gets(x[i].name); // x[i] ahol i >= 100 → UB / memóriakorrupció
```

Javítás:

```
#define MAX_PATIENTS 100
```

```
    scanf("%d",&n);
    if (num + n > MAX_PATIENTS) {
        printf("Error: the database can store at most %d records.\n",
                MAX_PATIENTS);
        return;
    }
    while (getchar() != '\n');
    endIndex=n+num;

    for(i=num; i<endIndex; i++)
    {
```

3e. Egész szám túlcsordulás — telefonszám tárolása int-ben

A telefonszám `int` típusú mezőben van tárolva. Egy `int` 32 bites rendszereken legfeljebb 2 147 483 647 értéket tárolhat. A legtöbb telefonszám (pl. +36301234567) meghaladja ezt a határt, ami **egész szám túlcsordulást (integer overflow)** okoz: a scanf az értéket truncálja, és a tárolt szám nem egyezik meg a beírt számmal.

```
    int cabin,phone,age;  // phone: int → max 2 147 483 647

// Beírt érték: 06301234567 (11 számjegy)
// Tárolt érték: -1 vagy más implementáció-függő truncált érték
```

Javítás — string vagy long long típus használata:

```
// 1. lehetőség: karakterlánc (megőrzi a vezető nullákat is):
    char phone[16];
        printf("Enter phone number = ");
        fgets(patients[i].phone, sizeof(patients[i].phone), stdin);

// 2. lehetőség: long long (64 bites egész):
    long long phone;
        scanf("%lld", &patients[i].phone);
```

---

4. Elavult függvények és rossz gyakorlatok

gets() — eltávolítva a C11 szabványból

A `gets()` függvény a C programozás egyik leghírhedtebb biztonsági kockázata. Az ISO C11 szabvány (2011) teljesen eltávolította a standard könyvtárból. Korábbi verzióiban jelen volt ugyan,

de a CERT C Coding Standard és a CWE-120 (Buffer Copy without Checking Size of Input) is külön azonosítóval jelöli meg mint kritikus sérülékenységet.

A függvény **nem ellenőrzi** a célbuffer méretét, és addig olvas, amíg sortörést vagy EOF-ot nem talál. Ez azt jelenti, hogy bármilyen hosszú bemenettel képes a cél memóriaterületén túlra írni.

```
// Kódban előforduló helyek (add(), edit(), search()):
C original.c                                                    U
gets(x[i].name); // x[i] ahol i >= 100 → UB / memóriakorrupció
gets(x[i].disease);
gets(x[p].name);
gets(x[p].disease);
gets(u);
gets(u);

// Biztonságos csere:
91          fgets(patients[i].name, sizeof(patients[i].name), stdin);
92          patients[i].name[strcspn(patients[i].name, "\n")] = '\0'; // sortörés eltávolítása
```

fflush(stdin) — meghatározatlan viselkedés Linuxon

A `fflush()` függvény az ISO C szabvány szerint kizárólag **kimeneti adatfolyamokra** (output streams) definiált. A `stdin` bemeneti adatfolyamon hívva a viselkedés **meghatározatlan (undefined behavior)**. A kódban ez a hívás összesen 17 helyen szerepel.

Egyes platformokon (pl. MSVC/Windows) a `fflush(stdin)` véletlenszerűen működhet, de a Linux/glibc implementáción *nincs semmilyen hatása* — vagyis a scanf/gets közti bemeneti maradékok nem törlődnek, ami váratlan viselkedéshez vezet a gets() hívások előtt.

```
// A kódban tipikusan így jelenik meg:
83              gets(x[i].name);
84              fflush(stdin);

// Helyes megoldás: scanf után a newline-t fogyasszuk el:
97          scanf("%d",&patients[i].age);
98          while (getchar() != '\n'); // bemeneti puffer kiürítése
```

strcpy() — ellenőrzetlen másolás a del()-ben

A `del()` függvény a rekordok eltolásához `strcpy()`-t használ. Az `strcpy()` nem veszi figyelembe a célbuffer méretét — bár itt a cél és forrás azonos méretű struct mezők (name[30] → name[30]), a függvény önmagában nem nyújt védelmet méretkülönbség esetén. Biztonságosabb alternatíva a `strncpy()` vagy a teljes struct másolása értékadással:

```
// Eredeti:
354                 strcpy(x[f].name,x[f+1].name);

// Biztonságosabb struct értékadással:
```

```
352            while(f < num - 1) // az utolsó valid index: num-2 → f+1 = num-1
353            {
354                patients[f] = patients[f+1]; // az egész struct-ot egy lépésben másolja
355                f++;
356            }
```

```
// maradék helyeken a "Cleared" 8 byte (7 karakter + \0), a cél buffer 30 byte,
nincs túlcsordulási kockázat
```

---

5. Memóriaszivárgás vizsgálata

Valgrind futtatás eredményei

A program `valgrind --leak-check=full` parancson keresztül futtatva a következő összesített eredményt adta:

```
razor@ReBorn:/mnt/e/source/repos/SZTE/Informacio_bizt/people$ valgrind --leak-check=full --track-origins=yes ./o1
==2139== Memcheck, a memory error detector
==2139== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==2139== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==2139== Command: ./o1
==2139==
========== Patient Recording System ===========
==2139== Conditional jump or move depends on uninitialised value(s)
==2139==    at 0x109489: main (in /mnt/e/source/repos/SZTE/Informacio_bizt/people/o1)
==2139==  Uninitialised value was created by a stack allocation
==2139==    at 0x109309: main (in /mnt/e/source/repos/SZTE/Informacio_bizt/people/o1)
==2139==
```

```
==2139== HEAP SUMMARY:
==2139==     in use at exit: 2,048 bytes in 2 blocks
==2139==   total heap usage: 4 allocs, 2 frees, 3,032 bytes allocated
==2139==
==2139== LEAK SUMMARY:
==2139==    definitely lost: 0 bytes in 0 blocks
==2139==    indirectly lost: 0 bytes in 0 blocks
==2139==      possibly lost: 0 bytes in 0 blocks
==2139==    still reachable: 2,048 bytes in 2 blocks
==2139==         suppressed: 0 bytes in 0 blocks
==2139== Reachable blocks (those to which a pointer was found) are not shown.
==2139== To see them, rerun with: --leak-check=full --show-leak-kinds=all
==2139==
==2139== For lists of detected and suppressed errors, rerun with: -s
==2139== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

Heap memóriaszivárgás nincs — de az oka figyelemre méltó

A program **egyetlen** dinamikus memóriafoglalást sem végez: sem `malloc()`, sem `calloc()`, sem `realloc()` nem szerepel a kódban. Az összes adat globális statikus tömbben (`struct ad x[100]`) tárolódik, amit a fordító a program indulásakor a BSS szegmensben foglal le, és a folyamat befejezésekor automatikusan felszabadít az OS. Ezért a heap-szivárgás nulla.

Ez azonban **nem jelenti azt, hogy a memóriakezelés hibátlan**. A Valgrind összesen **1 hibát** talált (bemenetek megadása nélkül), amely az inicializálatlan `c` változóra (lásd 3b. fejezet) vezethetők vissza: a `while(c!=6)` feltétel minden egyes iteráció előtt meghatározatlan értéket olvas ki.

A fread() bináris fájlolvasás kockázatai

A `read()` függvény bináris formátumban tölti be az adatokat `fread(x, sizeof(struct ad), 100, fp)` hívással. Ez több kockázatot rejt magában:

- **Nem validált adat:** Ha a patient.txt fájlt külső eszközzel módosítják, a beolvasott mezők tetszőleges értékeket tartalmazhatnak (pl. túl hosszú name mező nullbájt nélkül, negatív kor, stb.).
- **Platformfüggő bináris formátum:** A struct padding és a mezőméretek fordítóverziótól, architektúrától és optimalizációs szinttől függnek — a bináris fájl nem hordozható.
- **Nem ellenőrzött visszatérési érték:** A fread() visszatérési értéke (ténylegesen beolvasott rekordok száma) ugyan num-ba kerül, de a fclose() hibakezelése hiányzik.

Javítási javaslat — text alapú CSV formátum:

```c
void read()
{
    FILE *fp = fopen("patient.txt", "r");
    if (fp == NULL) {
        fp = fopen("patient.txt", "w");
        fclose(fp);
        printf("File does not exist, I JUST CREATED IT, exiting...\n\n\n");
        return;
    }

    num = 0;
    while (fscanf(fp, "%29[^,],%29[^,],%d,%lld,%d\n",
                patients[num].name, patients[num].disease,
                &patients[num].age, &patients[num].phone,
                &patients[num].cabin) == 5 && num < MAX_PATIENTS) {
        num++;
    }
    fclose(fp);
}

void write()
{
    FILE *fp = fopen("patient.txt", "w");
    if (fp == NULL) { printf("Error"); exit(1); }

    for (int i = 0; i < num; i++) {
        fprintf(fp, "%s,%s,%d,%lld,%d\n",
                patients[i].name, patients[i].disease,
                patients[i].age, patients[i].phone, patients[i].cabin);
    }
    fclose(fp);
}
```

Így viszont a névben, betegségben nem írható vessző, különben A CSV formátum megtörik.

6. Jól megoldott részek

- **fopen() NULL-ellenőrzés a write()-ban:** A fájlírás előtt a kód ellenőrzi, hogy az fopen() NULL-t adott-e vissza, és hiba esetén exit(1)-gyel leáll. Ez helyes hibakezelési minta, amelyet a read() ágban is alkalmaztak.
- **Funkciók szerinti szétbontás:** A program feladatait külön függvényekbe szervezték (add, view, search, edit, del, read, write), ami elvben helyes megközelítés és megkönnyíti az elkülönített tesztelést.
- **Egész szám index határ-ellenőrzés a search()-ben (h==1):** A serial szám alapú keresésnél az if(s<num) feltétel megakadályozza, hogy érvénytelen indexre hivatkozzanak.
- **Fájl automatikus létrehozása:** Ha a patient.txt nem létezik, a program automatikusan létrehozza, és tájékoztatja a felhasználót — ez felhasználóbarát viselkedés, és megakadályozza a néma indítási hibát.
- **Érvénytelen menü opció lekezelése:** A main() else ága üzenetet jelenít meg érvénytelen bemenet esetén, ahelyett hogy csendben továbblépne.

Összefoglaló

Kategória	Sérülékenység/Probléma	Súlyosság
Fordítói figyelmeztetés	gets() implicit deklaráció	🔴 Kritikus
Fordítói figyelmeztetés	void read() return 0;	🟡 Közepes
Fordítói figyelmeztetés	Inicializálatlan/unused változók	🟡 Közepes
Sérülékenység	Buffer overflow (gets)	🔴 Kritikus
Sérülékenység	Inicializálatlan c (UB, Valgrind 833 hiba)	🔴 Kritikus
Sérülékenység	OOB írás a del()-ben (x[num])	🔴 Kritikus
Sérülékenység	Tömbkapacitás túllépés (>100 rekord)	🔴 Kritikus
Sérülékenység	Telefonszám integer overflow	🟠 Magas
Elavult függvény	gets() — C11-ből eltávolítva	🔴 Kritikus
Elavult függvény	fflush(stdin) — UB Linuxon	🟠 Magas
Kód minőség	Kriptikus globális változónevek	🟡 Közepes
Kód minőség	show() deklarált, de definiálatlan	🟢 Alacsony
Memória	Nincs heap-szivárgás (statikus tömb)	✅ Rendben